Embedding multisets of vectors

A geometric method by twofold barycentric subdivision

June 24, 2026

Abstract

We are given n vectors in $\mathbb{R}^k$ with *no preferred order* — a *multiset* — and we want to summarise them by a single point of some $\mathbb{R}^N$ in a way that (i) ignores the order, (ii) loses no information, and (iii) varies continuously as the vectors move. These three demands pull against one another, and the naive fix (sorting) fails the third. We describe a method that meets all three. Its engine is a classical piece of geometry — the *barycentric subdivision* of a simplex — applied *twice*. Order-invariance and continuity will be almost immediate. The real work, and the heart of this note, is the remaining demand, *injectivity*: that distinct multisets really do land on distinct points. We give a geometric proof, and explain why a *single* subdivision is provably not enough.

Contents

1 The problem

1.1 Multisets, and how we shall write them

A *multiset* is a collection in which repeats are allowed and counted, but order is ignored. Thus $\{3, 3, 5\}$ and $\{5, 3, 3\}$ are the *same* multiset, while $\{3, 5\}$ and $\{3, 3, 5\}$ differ. We are interested in multisets of n vectors drawn from $\mathbb{R}^k$.

We record such a multiset as a matrix. Stack the n vectors as the *columns* of a $k \times n$ matrix

$$M \;=\; \big[\; \mathbf{v}_1 \;\big|\; \mathbf{v}_2 \;\big|\; \cdots \;\big|\; \mathbf{v}_n \;\big], \qquad \mathbf{v}_j \in \mathbb{R}^k.$$

Row i of M thus holds the i-th coordinate of every vector; column j holds the whole of vector $\mathbf{v}_j$. Because the vectors form a multiset, the *column order carries no meaning*: re-ordering the columns gives a different matrix but the *same* multiset. Formally, the symmetric group S_n (the group of all $n!$ permutations of $\{1, \ldots, n\}$) acts on matrices by permuting columns,

$$(\sigma \cdot M) \text{ has } j\text{-th column } \mathbf{v}_{\sigma^{-1}(j)}, \qquad \sigma \in S_n,$$

and two matrices represent the same multiset exactly when one is obtained from the other by such a column permutation. We write $M \sim M'$ in that case.

Example 1.1. With $n = 3$, $k = 2$,

$$M = \begin{bmatrix} 4 & 1 & 4 \\ 0 & 2 & 5 \end{bmatrix} \quad \text{and} \quad M' = \begin{bmatrix} 1 & 4 & 4 \\ 2 & 5 & 0 \end{bmatrix}$$

represent the same multiset $\big\{\,(4,0), (1,2), (4,5)\,\big\}$: M' is M with its columns cyclically shifted. Note that the two columns $(4, 0)$ and $(4, 5)$ *share their first entry* but are different vectors; multisets happily contain such near-collisions, and our method must keep them apart.

1.2 What we want to build

We seek a map f that sends a multiset M (equivalently, its $\sim$-class) to a point $f(M) \in \mathbb{R}^N$ for some fixed N, with three properties.

(P1) Permutation-invariance. $f(M) = f(M')$ whenever $M \sim M'$. (The output depends only on the multiset, not on how we happened to list it.)

(P2) Injectivity. $f(M) = f(M')$ *only* when $M \sim M'$. (Distinct multisets give distinct outputs — no information is lost. A map that is both (P1) and (P2) is called an *embedding*.)

(P3) Continuity. f is continuous: nudging the vectors a little moves $f(M)$ only a little.

Remark 1.2 (why all three at once is awkward)**.** Any one or two of these is easy. The difficulty is all three together. The obvious route to (P1) and (P2) is to *sort*: agree on some fixed rule for putting the columns in a canonical order (say, sort them lexicographically), and read off the sorted matrix. Sorting certainly forgets the original order (P1) and is reversible onto multisets (P2). But it is *not* continuous: as the vectors move and two of them swap places in the sorted order, the canonical matrix jumps. Picture two columns of nearly equal value sliding past each other — at the instant they cross, the "sort" snaps them into the opposite arrangement. So sorting buys (P1)+(P2) at the cost of (P3). The art is to obtain the invariance of sorting *without* its kinks.

Remark 1.3 (a harmless normalisation)**.** It is convenient, and costs nothing, to also ask for *positive homogeneity*: $f(\lambda M) = \lambda f(M)$ for scalars $\lambda > 0$. Geometrically this lets us think in terms of *cones* and *rays* rather than fussing over overall scale, and the construction below provides it automatically. The reader may ignore it on a first pass.

1.3 The plan, and where the difficulty lies

The method has the following shape, and the rest of the document fills it in.

- We first treat the easy case $k = 1$ — a multiset of n *numbers* — because it already contains the central geometric idea in its simplest form (Section 3).
- That idea is the *barycentric subdivision* of a simplex, which we define from scratch and which turns the discontinuous "sort" into a continuous gadget.
- For genuine vectors ($k \geq 2$) the rows interact, and we shall see that applying the idea once *per row* is *not enough* for injectivity — a concrete pair of multisets collides. This failure, and its cure by a *second* subdivision, is the crux (Sections 4 onward).
- Finally a *hashing* step turns the geometric data into a point of $\mathbb{R}^N$.

Throughout, (P1) and (P3) will be easy to see as we go; the burden of proof is (P2), injectivity, and in particular *why two subdivisions are needed and one is not.* That is the non-trivial content.

2 The geometric idea: barycentric subdivision

Before any multisets, we recall the one piece of geometry the whole method runs on. It is elementary and classical; we set it up carefully because everything later is an echo of it.

2.1 The simplex and barycentric coordinates

Fix n points $\mathbf{e}_1, \ldots, \mathbf{e}_n$ in general position (no three on a line, etc.). Their *simplex* is the solid region they span,

$$\Delta \;=\; \Big\{ \textstyle\sum_{i=1}^{n} \alpha_i \, \mathbf{e}_i \;:\; \alpha_i \geq 0,\ \sum_i \alpha_i = 1 \Big\}.$$

For $n = 3$ this is a (filled) triangle; for $n = 4$ a tetrahedron. The numbers $(\alpha_1, \ldots, \alpha_n)$ are the *barycentric coordinates* of the point $\sum_i \alpha_i \mathbf{e}_i$. The defining fact of a simplex is that *barycentric coordinates are unique*: each point of Δ is $\sum_i \alpha_i \mathbf{e}_i$ for exactly one choice of weights $\alpha_i \geq 0$ summing to 1. A weight α_i is zero exactly when the point lies on the face opposite $\mathbf{e}_i$.

The *barycentre* of a set of vertices is their average. The barycentre of the whole simplex is $\mathbf{g} = \frac{1}{n}(\mathbf{e}_1 + \cdots + \mathbf{e}_n)$ (all weights equal); the barycentre of an edge $\mathbf{e}_i\mathbf{e}_j$ is its midpoint $\frac{1}{2}(\mathbf{e}_i + \mathbf{e}_j)$, and so on.

2.2 Subdividing the simplex

Barycentric subdivision cuts Δ into smaller simplices using these barycentres. The recipe, for our purposes, is cleanest stated through *orderings*. Given an ordering σ of the vertices — say $\mathbf{e}_{\sigma(1)}, \mathbf{e}_{\sigma(2)}, \ldots, \mathbf{e}_{\sigma(n)}$ — form the running barycentres of its *prefixes*:

$$\mathbf{f}_r^\sigma \;=\; \frac{1}{r}\big(\mathbf{e}_{\sigma(1)} + \mathbf{e}_{\sigma(2)} + \cdots + \mathbf{e}_{\sigma(r)}\big), \qquad r = 1, 2, \ldots, n. \tag{1}$$

So $\mathbf{f}_1^\sigma = \mathbf{e}_{\sigma(1)}$ is a vertex, $\mathbf{f}_2^\sigma$ is an edge-midpoint, …, and $\mathbf{f}_n^\sigma = \mathbf{g}$ is the centre of the whole simplex (the same point for every σ). These n points span a small simplex, the *cell* C_σ. As σ ranges over all $n!$ orderings, the cells C_σ tile Δ without overlap (Figure 1).

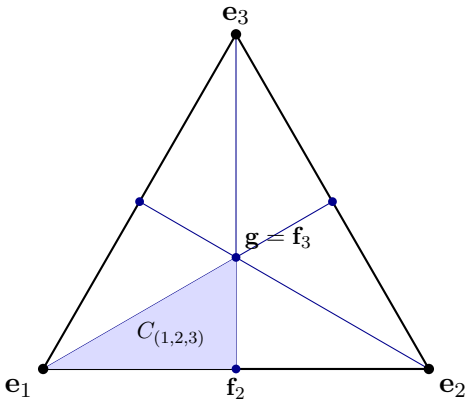

Figure 1: Barycentric subdivision of a triangle ($n = 3$): the three medians cut it into $3! = 6$ cells, one per ordering of the vertices. The shaded cell $C_{(1,2,3)}$ has vertices $\mathbf{f}_1 = \mathbf{e}_1$, $\mathbf{f}_2 = \frac{1}{2}(\mathbf{e}_1+\mathbf{e}_2)$ and $\mathbf{f}_3 = \mathbf{g} = \frac{1}{3}(\mathbf{e}_1+\mathbf{e}_2+\mathbf{e}_3)$. The centre $\mathbf{g}$ is shared by all six cells.

2.3 The one fact we shall use again and again

Here is the property that makes the subdivision useful. It says that locating a point among the cells is a *continuous* substitute for sorting.

Proposition 2.1 (unique cell, unique coordinates)**.** *Let $p = \sum_i \alpha_i \mathbf{e}_i$ be a point of Δ, and let σ be an ordering that sorts its weights into non-increasing order, $\alpha_{\sigma(1)} \geq \alpha_{\sigma(2)} \geq \cdots \geq \alpha_{\sigma(n)}$. Then p lies in the cell C_σ, and its barycentric coordinates there are, up to the factor r, the* gaps *$\alpha_{\sigma(r)} - \alpha_{\sigma(r+1)}$ between consecutive sorted weights:*

$$\beta_r \;=\; r\,\big(\alpha_{\sigma(r)} - \alpha_{\sigma(r+1)}\big) \;\geq 0, \qquad (\alpha_{\sigma(n+1)} := 0), \tag{2}$$

so that $p = \sum_{r=1}^{n} \beta_r\, \mathbf{f}_r^\sigma$ with $\beta_r \geq 0$ and $\sum_r \beta_r = 1$. If the sorting order is strict *the cell is unique; if two weights are tied, $\alpha_{\sigma(r)} = \alpha_{\sigma(r+1)}$, then the corresponding gap, and so β_r, is* zero*, and p lies on a wall shared by the two cells that differ only by swapping those two ranks.*

Why this is true. That (2) reproduces p is the discrete "summation by parts" identity $\sum_i \alpha_i \mathbf{e}_i = \sum_r r(\alpha_{\sigma(r)} - \alpha_{\sigma(r+1)})\mathbf{f}_r^\sigma$, which one checks by expanding $\mathbf{f}_r^\sigma$ from (1) and collecting the coefficient of each $\mathbf{e}_{\sigma(r)}$ (it telescopes back to $\alpha_{\sigma(r)}$). The gaps are ≥ 0 precisely because the weights were sorted, which is what places p in C_σ; and a tie makes a gap vanish, putting p on the boundary face where that barycentre drops out. Uniqueness of the β_r is uniqueness of barycentric coordinates in the cell. □

The point of Proposition 2.1. "Which cell?" is decided by the *order* of the weights — a discrete, sorting-like datum. But the *coordinates* β_r vary continuously, and exactly when the order is ambiguous (a tie) the ambiguous coordinate is 0, so the point sits cleanly on a shared wall and nothing jumps. This is how a subdivision launders the discontinuity out of sorting: *the place where the cell-label is ambiguous is exactly the place where it does not matter.*

3 Warm-up: a multiset of numbers ($k = 1$)

Take the simplest case first: $k = 1$, so each "vector" is a single number and the data is a multiset of n reals $x_1, \dots, x_n$, written as the single row $M = [\, x_1 \ \cdots \ x_n \,]$. This already contains the heart of the idea, and solving it cleanly tells us precisely what will be *missing* when $k \geq 2$.

3.1 The same picture, in a cone

Proposition 2.1 was about a simplex (weights summing to 1). The identical statement holds, even more simply, in the positive "orthant" $\{\, x \in \mathbb{R}^n : x_i \geq 0 \,\}$ if we drop the sum-to-one condition. Sort the entries of x into non-increasing order, $x_{(1)} \geq x_{(2)} \geq \cdots \geq x_{(n)} \geq 0$, and form the *prefix vectors*

$$\mathbf{u}_r \ = \ \mathbf{e}_{(1)} + \mathbf{e}_{(2)} + \cdots + \mathbf{e}_{(r)} \qquad \text{(the indicator of the top-}r\text{ positions)},$$

where $\mathbf{e}_{(s)}$ is the standard basis vector for the position holding the s-th largest entry. Then summation by parts gives, just as in (2),

$$x \ = \ \sum_{r=1}^{n} \delta_r \, \mathbf{u}_r, \qquad \delta_r = x_{(r)} - x_{(r+1)} \ \geq 0, \quad (x_{(n+1)} := 0). \tag{3}$$

The coefficients δ_r are the *gaps* between consecutive sorted values; $\delta_n = x_{(n)}$ is the smallest entry, attached to the all-ones vector $\mathbf{u}_n = \mathbf{e}_1 + \cdots + \mathbf{e}_n$. (Geometrically the $\mathbf{u}_r$ are just the face-barycentres $\mathbf{f}_r$ of (1) un-normalised by the factor $1/r$; nothing has changed but the bookkeeping of scale.)

Example 3.1. For $x = (2, 9, 2, 6)$, sorting gives $9 \geq 6 \geq 2 \geq 2$, so the gaps are $\delta_1 = 9{-}6 = 3$, $\delta_2 = 6{-}2 = 4$, $\delta_3 = 2{-}2 = 0$, $\delta_4 = 2$. The prefixes are: top-1 $= \{\text{the } 9\}$, top-2 $= \{9, 6\}$, top-3 $= \{9, 6, 2\}$, top-4 $=$ everything. The vanishing gap $\delta_3 = 0$ records the tie between the two 2's — and note it makes the (ambiguous: *which* 2?) top-3 prefix irrelevant, since it is multiplied by 0.

3.2 Reading off the invariant, and why all three properties hold

To turn (3) into something that forgets the labelling, observe what survives relabelling. Permuting the n numbers permutes the standard basis vectors, hence relabels *which* positions sit in each prefix — but it changes neither the gaps δ_r nor the *sizes* of the prefixes. And for a multiset of bare numbers, a prefix carries no information beyond its size: any two top-r sets are interchangeable. So the honest, label-free record of x is simply the list of pairs

$$\{\, (\delta_r, \ r) \ : \ r = 1, \dots, n \,\} \qquad \text{(gap, prefix-size)},$$

which is no more and no less than the sorted gaps. Let this be $f(M)$ (we shall hash it into a single $\mathbb{R}^N$ point later; for now its content is what matters).

- **(P1) Invariance** is immediate: gaps and sizes do not depend on how the numbers were listed.
- **(P2) Injectivity** holds because the data is reversible: from the gaps we rebuild the sorted values ($x_{(r)} = \delta_r + \delta_{r+1} + \cdots + \delta_n$), and the sorted values *are* the multiset.

- **(P3) Continuity** holds because each gap δ_r is a continuous function of the multiset (order statistics move continuously), the sizes r are fixed integers, and — crucially — at any tie the moving gap is 0, so no quantity jumps. This is exactly the laundering of Proposition 2.1: we never had to commit to *which* tied number was "larger".

So for $k = 1$ a *single* barycentric subdivision already does everything.

3.3 The crack that opens at $k \geq 2$

Look hard at why injectivity was so cheap above: a prefix mattered only through its *size*, so "canonicalising" it (forgetting the labels) lost nothing we needed. That is special to one row.

With $k \geq 2$ rows, the columns are shared: a single permutation reorders the columns of *every* row at once. Now a prefix in row i is a genuine *subset of the n columns*, and — because the same columns recur in every row — *which* columns it contains, not merely how many, can matter. Two multisets can agree row-by-row on all the gaps and all the prefix-sizes, yet differ in how the prefixes of *different rows line up over the shared columns*. Forgetting the column labels one row at a time would erase exactly that cross-row alignment.

Section 4 makes this precise and exhibits two genuinely different multisets that a one-subdivision-per-row scheme cannot tell apart. Repairing the damage is what the *second* subdivision is for.

4 Vectors, and why one subdivision is not enough

Now let $k \geq 2$. We apply the warm-up idea to *each row* of M and watch it fall short.

4.1 The first subdivision, applied row by row

Each row of M is a list of n numbers, so Section 3 applies to it verbatim. Sorting row i and decomposing as in (3) writes that row as a sum of gaps times prefixes. The novelty is purely in the bookkeeping of *which columns* a prefix contains, so we name these objects once and for all — they are the raw material of everything that follows.

Definition 4.1 (piece)**.** Fix a row i and a level $r \in \{1, \ldots, n-1\}$. Sorting row i singles out its *top-r prefix*: the set of the r columns carrying that row's largest r entries. The associated *piece* is the $k \times n$ matrix P that holds the 0/1 indicator of this prefix in row i and is zero in every other row. We say P *sits in row i* and has *size r*, and we attach to it the *gap* $\delta_r^{(i)} \geq 0$ of (3). There are exactly $m = k(n-1)$ pieces — $(n-1)$ per row — and, because the prefixes within a row are nested, a piece is pinned down by its $(row, size)$ alone.

A piece is thus one rung of a single row's prefix ladder, lifted into a $k \times n$ matrix so that pieces from *different* rows inhabit a common space and can be added. From here on, pieces (and sums of them) are the only objects we manipulate.

Example 4.2. Let $n = 2$, $k = 2$ and

$$A = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}.$$

Row $1 = (3, 0)$: top-1 prefix is column 1, gap $\delta_1^{(1)} = 3$. Row $2 = (0, 2)$: top-1 prefix is column 2, gap $\delta_1^{(2)} = 2$. As pieces (the all-ones size-2 prefixes carry the row minima, here 0, so we drop them):

$$\underbrace{\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}}_{\text{row-1 top, gap 3}} \qquad \underbrace{\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}}_{\text{row-2 top, gap 2}}.$$

The whole of A (above its row minima) is $3 \cdot (\text{first piece}) + 2 \cdot (\text{second piece})$.

So after the first subdivision the data is: for each row, a sorted list of *(gap, prefix)* pairs, the prefixes being subsets of the n shared columns.

4.2 The naive repair, and what it really quotients by

To make this permutation-invariant we must forget the column labels. The warm-up did so by keeping only prefix *sizes*. If we do the same here — canonicalise each row's prefixes *on their own*, recording each only up to relabelling its columns — we are quotienting each row *separately* by its own copy of S_n. That is the product group $S_n \times \cdots \times S_n$ (k copies), allowing a *different* column permutation in every row.

But the symmetry we are actually entitled to is far smaller: a multiset permits *one* permutation applied to *all* rows simultaneously — the *diagonal* copy of S_n inside the product. Quotienting by the whole product is coarser; it glues together matrices that are *not* column rearrangements of one another. We have thrown away the cross-row alignment warned of in Section 3.3. The next example shows this is fatal.

4.3 Two multisets the naive scheme cannot separate

Example 4.3 (the collision)**.** Keep $n = 2$, $k = 2$ and set, alongside A above,

$$A = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}, \qquad B = \begin{bmatrix} 3 & 0 \\ 2 & 0 \end{bmatrix}.$$

These are *different multisets*:

$$A = \{\, (3,0),\, (0,2) \,\}, \qquad B = \{\, (3,2),\, (0,0) \,\},$$

and no reordering of two columns turns one into the other (A has two vectors each with a single zero entry; B has the vector $(3,2)$ and the zero vector).

Yet run the naive per-row scheme. In *both* matrices:

- row 1 is $(3,0)$ — gap 3, one top prefix of size 1, row-min 0;
- row 2 is $(2,0)$ or $(0,2)$ — either way gap 2, one top prefix of size 1, row-min 0.

Row by row, after forgetting which column is which, the recorded data is identical: {row 1: gap 3, size 1}, {row 2: gap 2, size 1}. The naive scheme returns the *same* answer for A and B. It is **not injective**. The single fact it could not see is whether the two rows' top columns are the *same* column (as in B) or *different* columns (as in A) — precisely a statement about how the rows align over the shared labels, which no amount of per-row canonicalisation can recover.

4.4 The cure, in one sentence

The defect is that we canonicalised *row-local* objects. To respect only the diagonal S_n, we must canonicalise objects that *span all the rows at once*, so that a single column permutation acts on all of them together. The cheapest such objects are *sums of pieces drawn from different rows*: a single $k \times n$ matrix in which several rows are simultaneously populated.

Watch how this already rescues Example 4.3. Add the two pieces of each matrix into one cross-row matrix:

$$A: \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad B: \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}.$$

Now canonicalise by sorting the *columns* of each (one permutation for the whole matrix). The columns of A's matrix are $(1,0)$ and $(0,1)$; of B's, $(1,1)$ and $(0,0)$. These are different as multisets of columns, so the canonical forms *differ*: the cross-row sum sees what the per-row data could not — namely whether the two top entries share a column. A single matrix can only be tidied by a single column permutation, which is exactly the diagonal symmetry we are allowed.

This is the whole reason for a *second* subdivision: it is the systematic, continuous way to manufacture and weight these cross-row matrices. We turn to it next.

5 The second subdivision, and the finished method

5.1 Pooling the pieces

After the first subdivision (one per row) we hold a bag of $m = k(n-1)$ *(gap, piece)* pairs — $(n-1)$ proper prefixes from each of the k rows. (The k all-ones, size-n prefixes are fully symmetric: a column permutation fixes each of them. They carry no alignment information, so we set them aside as k *anchors*, namely the k row minima $m_1, \dots, m_k$, which are plainly permutation-invariant numbers; we record them directly and forget them for now.)

Crucially, this bag is *itself* a point in a cone — the cone spanned by the m pieces, with the gaps as coordinates:

$$(\text{balance of } M) \;=\; \sum_{\text{pieces } P} \delta_P \, P.$$

So we may apply the *very same* barycentric subdivision a second time, now to this m-piece cone, with the pieces playing the role the standard basis vectors $\mathbf{e}_i$ played before.

5.2 The second subdivision

Sort the m pieces into non-increasing gap order, $\delta_{(1)} \geq \delta_{(2)} \geq \cdots \geq \delta_{(m)}$, and (Proposition 2.1, in the cone form (3), applied to the pieces) form the running sums

$$L_s \;=\; P_{(1)} + P_{(2)} + \cdots + P_{(s)}, \qquad \alpha_s \;=\; \delta_{(s)} - \delta_{(s+1)} \;\geq 0, \tag{4}$$

for $s = 1, \dots, m$ (with $\delta_{(m+1)} := 0$), so that, by summation by parts, balance $= \sum_t \delta_{(t)} P_{(t)} = \sum_s \alpha_s L_s$. Each L_s is a single $k \times n$ matrix of non-negative integers — a *cross-row* object, exactly the kind Section 4 said we needed: it is the running total of the highest-gap pieces, and because those pieces come from *different rows* it populates several rows at once. We call L_s a *cumulative matrix*.

Remark 5.1. This is genuinely the same operation as the first subdivision — "sort, then take running barycentres" — but now the things being sorted are whole pieces, not numbers within a row, and the running sums knit the rows together. The two applications differ only in what they are applied to: *within each row* first, then *across the pooled pieces*.

Example 5.2 (the second subdivision, worked)**.** Take $n = 3$, $k = 2$ and

$$M = \begin{bmatrix} 6 & 1 & 2 \\ 4 & 10 & 1 \end{bmatrix}.$$

Both row minima equal 1, so the anchors are $m_1 = m_2 = 1$; subtracting them leaves the balance $\left[\begin{smallmatrix} 5 & 0 & 1 \\ 3 & 9 & 0 \end{smallmatrix}\right]$. The first subdivision (Definition 4.1) produces four pieces, shown here with their gaps:

$$\underbrace{\left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \end{smallmatrix}\right]}_{\text{row 1, size 1; gap 4}} \qquad \underbrace{\left[\begin{smallmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \end{smallmatrix}\right]}_{\text{row 1, size 2; gap 1}} \qquad \underbrace{\left[\begin{smallmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]}_{\text{row 2, size 1; gap 6}} \qquad \underbrace{\left[\begin{smallmatrix} 0 & 0 & 0 \\ 1 & 1 & 0 \end{smallmatrix}\right]}_{\text{row 2, size 2; gap 3}}$$

Pooling and sorting by gap ($6 \geq 4 \geq 3 \geq 1$) orders the pieces as $P_{(1)}, \dots, P_{(4)}$, and the running sums (4) are the cumulative matrices

$$\underbrace{\left[\begin{smallmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]}_{L_1,\ \alpha_1=2} \qquad \underbrace{\left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]}_{L_2,\ \alpha_2=1} \qquad \underbrace{\left[\begin{smallmatrix} 1 & 0 & 0 \\ 1 & 2 & 0 \end{smallmatrix}\right]}_{L_3,\ \alpha_3=2} \qquad \underbrace{\left[\begin{smallmatrix} 2 & 0 & 1 \\ 1 & 2 & 0 \end{smallmatrix}\right]}_{L_4,\ \alpha_4=1}$$

with weights $\alpha_s = \delta_{(s)} - \delta_{(s+1)}$ read off $\delta = (6, 4, 3, 1)$. As a check, $\sum_s \alpha_s L_s$ rebuilds the balance — e.g. its $(2,2)$ entry is $2{\cdot}1 + 1{\cdot}1 + 2{\cdot}2 + 1{\cdot}2 = 9$. Two features to note for later: L_s grows by exactly one piece at each step, and the richest matrix L_4 has carried integer counts up to 2 — the "staircase" that Section 6 will read back to recover everything.

5.3 Canonicalise, and assemble the output

Finally make the result blind to the column labelling. Canonicalise each cumulative matrix L_s by sorting its columns into a fixed standard order (say lexicographically); call the result $\widehat{L}_s$. Because a single permutation is applied to the whole matrix, this respects exactly the diagonal S_n. The output of the method is the collection

$$f(M) \;=\; \Big(\underbrace{m_1, \ldots, m_k}_{\text{anchors}} \;;\; \big\{\, (\widehat{L}_s,\ \alpha_s) \;:\; \alpha_s > 0 \,\big\} \Big). \tag{5}$$

(Pairs with $\alpha_s = 0$ contribute nothing and are dropped.) A final *hashing* step, Section 7, compresses this into a single point of $\mathbb{R}^N$; but (5) already holds all the information, so we argue about it directly.

5.4 Invariance, continuity and homogeneity are immediate

As promised, the easy properties cost almost nothing.

Proposition 5.3 ((P1), (P3) and homogeneity)**.** *The map* (5) *is permutation-invariant, continuous, and positively homogeneous of degree* 1*: $f(\lambda M) = \lambda f(M)$ for every $\lambda > 0$. The last point settles Remark 1.3.*

Proof. (P1) A column permutation σ relabels the columns of every row identically, hence permutes the columns of every piece, hence of every cumulative matrix L_s, by the same σ; sorting the columns ($L_s \mapsto \widehat{L}_s$) erases it. The gaps, and therefore the weights α_s and the anchors m_i, are computed from sorted values and are unchanged by σ. So $f(\sigma \cdot M) = f(M)$.

(P3) Both subdivisions are instances of Proposition 2.1, whose coordinates (gaps, then the α_s) are continuous, with the saving grace that whenever an ordering is ambiguous — two tied row-values, or two tied gaps — the corresponding coordinate is 0. A cumulative matrix L_s whose existence depends on breaking such a tie is therefore weighted by $\alpha_s = 0$ and is dropped, so the (discontinuous) choice of how to break the tie never reaches the output. Hence f is continuous. (The anchors m_i are continuous as minima.)

Homogeneity. Scaling $M \mapsto \lambda M$ with $\lambda > 0$ multiplies every entry, hence every row minimum ($m_i \mapsto \lambda m_i$) and every gap ($\delta \mapsto \lambda\delta$, so $\alpha_s \mapsto \lambda\alpha_s$), by λ — but it changes no *order*. So the sort orders, the pieces, and the cumulative matrices L_s, being purely combinatorial, are untouched; each $\widehat{L}_s$ is identical, while every coordinate α_s and anchor m_i scales by λ. Thus the whole output (5), and its hash (6), scales by λ. This is exactly why it paid to work throughout with *cones and rays* rather than normalising: the combinatorial skeleton (which cells, which pieces) is scale-free, and only the coordinates carry the scale. □

That disposes of (P1) and (P3). Everything now turns on (P2).

6 Injectivity

This is the substance of the note. We show that the output (5) determines the multiset. Two simple observations do all the work; the second subdivision earns its keep in both.

6.1 A cumulative matrix remembers its pieces

The pieces summed into a cumulative matrix are not lost in the sum — they can be read straight back off. The reason is that the pieces from a given row are *nested*: in row i the contributing prefixes are some of top-1 $\subset$ top-2 $\subset \cdots$, so the count in column c simply records how many of them reach that column.

Lemma 6.1 (recovery)**.** *From a cumulative matrix L one can recover exactly which pieces were summed to build it. Concretely, in each row i the sets of columns*

$$\{\, c : \; L_{i,c} \geq t \,\}, \qquad t = 1, 2, 3, \ldots$$

are precisely the contributing prefixes of row i, listed from largest t (smallest prefix) upward.

Proof. Row i of L is a sum of indicators of nested prefixes $Q_1 \subset Q_2 \subset \cdots \subset Q_a$ (those present in row i). The entry in column c counts how many Q_b contain c. Because the Q_b are nested, the columns with entry $\geq t$ are exactly those lying in the t-th largest of them, i.e. in Q_{a-t+1}. So thresholding at $t = 1, 2, \ldots$ returns $Q_a, Q_{a-1}, \ldots, Q_1$ — all the contributing prefixes, columns and all. □

Example 6.2. Suppose a row of some L reads $(2, 0, 3, 1)$. Columns with entry ≥ 1 are $\{1, 3, 4\}$; with entry ≥ 2, $\{1, 3\}$; with entry ≥ 3, $\{3\}$; nothing reaches 4. So this row was built from the three nested prefixes $\{3\} \subset \{1, 3\} \subset \{1, 3, 4\}$ — the top-1, top-2 and top-3 sets of that row — and we have recovered them, columns included, from the row alone.

Two consequences will be used repeatedly.

Corollary 6.3 (pieces are named by (row, size))**.** *Within any row the contributing prefixes are nested, so there is* at most one *of each size. Hence every piece is unambiguously named by the pair (row, size), and this name is permutation-invariant (column sorting changes neither a piece's row nor its size).*

Corollary 6.4 (recovery survives canonicalisation)**.** *Sorting the columns of L permutes all rows by one permutation σ, which turns each recovered prefix Q into $\sigma(Q)$ but leaves its* size*, and the* nesting*, untouched. So from the canonical $\widehat{L}$ we still recover the piece-set — as a set of (row, size) names, exactly — and the actual columns up to the single permutation σ.*

6.2 The injectivity theorem

Theorem 6.5. *The map f of (5) is injective on multisets: if $f(M) = f(M')$ then $M \sim M'$.*

The proof separates cleanly into a *combinatorial* part that needs no column labels at all, and a single *alignment* step that fixes the labels once, using one matrix.

Proof. Suppose we are handed the output (5): the anchors $m_1, \ldots, m_k$ and the multiset of pairs $\{(\widehat{L}_s, \alpha_s) : \alpha_s > 0\}$. We reconstruct M up to a column permutation.

Step 1 (combinatorics, no labels needed). By Lemma 6.1 and Corollary 6.3, each $\widehat{L}_s$ reveals its piece-set as a set of (row, size) names — and these names are permutation-invariant, so there is *no ambiguity* here. The number of names is the size of the piece-set, which identifies the index s (recall L_s pooled the s highest-gap pieces). Ordering the survivors by this size gives a nested chain of piece-sets

$$\{P_{(1)}\} \subset \{P_{(1)}, P_{(2)}\} \subset \cdots,$$

each step naming the next piece in *gap order* — or, where several gaps coincide, the next *block* of equal-gap pieces, whose internal order is immaterial. Thus the gap-ordering of the pieces is recovered (up to that immaterial shuffling within tied blocks), as (row, size) names. Moreover the gaps themselves come back: since $\alpha_s = \delta_{(s)} - \delta_{(s+1)}$ we have $\delta_{(s)} = \sum_{t \geq s} \alpha_t$, so each piece's gap is known. With the anchors m_i, the sorted values of every row are now determined — as numbers attached to (row, size) slots, still with no column labels in sight.

Step 2 (one alignment, from one matrix). It remains to place the columns. Let L^* be the *richest* surviving cumulative matrix — the one whose piece-set is largest; it contains every piece that occurs (a piece with positive gap appears in L^*, one with zero gap is genuinely absent from M). By Corollary 6.4, choosing any column labelling that realises the canonical form $\widehat{L}^*$ fixes a

single permutation σ, and in this labelling L^* exhibits, in each row, the *whole* nested chain of that row's prefixes — that is, which column is the row's largest, second largest, and so on. One matrix thus pins the joint column ranking of *all* rows simultaneously, because all rows live in it together.

Assembly. Combine the two steps in the labelling σ: Step 1 supplies the value at each (row, rank) slot; Step 2 supplies which column occupies each rank in each row. Filling the values into the columns reconstructs the balance, and with the anchors, the matrix M — uniquely up to the one permutation σ, i.e. up to $\sim$. Hence the output determines M as a multiset, which is exactly injectivity. (Where a gap is zero the two tied columns are interchangeable, so the ambiguity is precisely a $\sim$-ambiguity and harms nothing.) □

What made it work. The combinatorics of Step 1 never needed column labels, because a piece is named by its *row and size* (Corollary 6.3) — both label-free. The *only* place labels matter is the alignment of columns across rows, and that is settled in one stroke by the single richest matrix L^*, which can be tidied by only *one* permutation. Producing such an all-rows-at-once matrix is exactly what the second subdivision did. Notice the contrast with Section 4: there we tried to align using row-local data and failed; here one cross-row matrix succeeds.

Example 6.6 (the theorem at work: $n = 4$, $k = 3$)**.** Take four vectors in $\mathbb{R}^3$, the columns of

$$M = \begin{bmatrix} 8 & 23 & 2 & 12 \\ 10 & 1 & 19 & 8 \\ 3 & 4 & 7 & 15 \end{bmatrix}.$$

The row minima are the anchors $m_1 = 2,\ m_2 = 1,\ m_3 = 3$; subtracting them leaves the balance $\left[\begin{smallmatrix} 6 & 21 & 0 & 10 \\ 9 & 0 & 18 & 7 \\ 0 & 1 & 4 & 12 \end{smallmatrix}\right]$. Sorting each row (first subdivision) gives its order, three prefixes, and gaps (columns named $v_1, \ldots, v_4$):

row, descending order	prefixes (gap)		
1 : $v_2{>}v_4{>}v_1{>}v_3$	$\{v_2\}$ (11),	$\{v_2v_4\}$ (4),	$\{v_2v_4v_1\}$ (6)
2 : $v_3{>}v_1{>}v_4{>}v_2$	$\{v_3\}$ (9),	$\{v_3v_1\}$ (2),	$\{v_3v_1v_4\}$ (7)
3 : $v_4{>}v_3{>}v_2{>}v_1$	$\{v_4\}$ (8),	$\{v_4v_3\}$ (3),	$\{v_4v_3v_2\}$ (1)

The three orders are genuinely different, so the alignment is non-trivial. Pooling the nine gaps and sorting, $\delta = (11, 9, 8, 7, 6, 4, 3, 2, 1)$, gives weights $\alpha_s = \delta_{(s)} - \delta_{(s+1)} = (2, 1, 1, 1, 2, 1, 1, 1, 1)$, and the running sums culminate in the richest cumulative matrix

$$L^* = \begin{bmatrix} 1 & 3 & 0 & 2 \\ 2 & 0 & 3 & 1 \\ 0 & 1 & 2 & 3 \end{bmatrix}, \qquad \left(L^*\right)_{i,c} = \#\{\text{row-}i \text{ prefixes containing column } c\}.$$

Now reconstruct, given only the output. Canonicalising L^* (sort its columns) yields $\widehat{L^*} = \left[\begin{smallmatrix} 0 & 1 & 2 & 3 \\ 3 & 2 & 1 & 0 \\ 2 & 0 & 3 & 1 \end{smallmatrix}\right]$, whose columns we can only call $c_1, \ldots, c_4$. Threshold each row (Lemma 6.1):

$$\begin{aligned} \text{row } 1 = (0,1,2,3): &\quad \{c_4\} \subset \{c_3, c_4\} \subset \{c_2, c_3, c_4\} \quad \Rightarrow\ c_4{>}c_3{>}c_2{>}c_1, \\ \text{row } 2 = (3,2,1,0): &\quad \{c_1\} \subset \{c_1, c_2\} \subset \{c_1, c_2, c_3\} \quad \Rightarrow\ c_1{>}c_2{>}c_3{>}c_4, \\ \text{row } 3 = (2,0,3,1): &\quad \{c_3\} \subset \{c_1, c_3\} \subset \{c_1, c_3, c_4\} \quad \Rightarrow\ c_3{>}c_1{>}c_4{>}c_2. \end{aligned}$$

That *one* matrix has handed back all three orders at once, consistently labelled — the whole alignment, with no cross-row guesswork. Step 1 is now routine: the weights rebuild the gaps, $\delta_{(s)} = \sum_{t \geq s} \alpha_t = (11, 9, 8, 7, 6, 4, 3, 2, 1)$, the (row, size) names pin each gap to its rung, cumulating gives the sorted balance values $(21, 10, 6, 0)$, $(18, 9, 7, 0)$, $(12, 4, 1, 0)$, and adding the anchors restores the rows. Pouring these values into the columns in the recovered orders reproduces M *exactly*, up to the relabelling $(v_1v_2v_3v_4) \mapsto (c_2c_4c_1c_3)$ — that is, up to $\sim$, as the theorem promises.

Example 6.7 (a column tie, and a dropped cumulative matrix)**.** Example 6.6 had distinct gaps, so every $\alpha_s > 0$ and every L_s was stored. A tie changes that, and it is worth seeing how the algorithm copes. Take $n = 3$, $k = 2$,

$$M = \begin{bmatrix} 5 & 2 & 8 \\ 4 & 4 & 1 \end{bmatrix}, \qquad \{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\} = \{(5,4), (2,4), (8,1)\},$$

in which $\mathbf{v}_1$ and $\mathbf{v}_2$ *tie in row* 2 (both 4) yet differ in row 1. Anchors $m_1 = 2,\ m_2 = 1$; balance $\left[\begin{smallmatrix} 3 & 0 & 6 \\ 3 & 3 & 0 \end{smallmatrix}\right]$. The first subdivision gives

$$\begin{array}{r|l} \text{row 1 :}\ \ v_2{>}v_1{>}v_3 & \{v_2\}\,(5),\quad \{v_2 v_1\}\,(2) \\ \text{row 2 :}\ \ v_1 \approx v_2{>}v_3 & \{v_1 \text{ or } v_2\}\,(0),\quad \{v_1 v_2\}\,(3) \end{array}$$

The row-2 top-1 prefix is *ambiguous* — which of the tied v_1, v_2? — and carries gap 0. Pooling and sorting, $\delta = (5, 4, 2, 0)$, so

$$\alpha = (1,\ 2,\ 2,\ \mathbf{0}) :$$

the last weight vanishes, and the cumulative matrix that would have rested on the ambiguous piece is *never stored*. The richest stored matrix is

$$L^* = \begin{bmatrix} 1 & 2 & 0 \\ 1 & 1 & 0 \end{bmatrix}.$$

Decoding it: row 1 reads $(1, 2, 0)$, giving the clean order $v_2{>}v_1{>}v_3$; row 2 reads $(1, 1, 0)$ — v_1 and v_2 sit at the *same* threshold level, with nothing above to separate them, so the output reports "v_1, v_2 are the top two of row 2, tied" and leaves their row-2 order *undetermined*. That is exactly right: they are equal there. Yet nothing is lost — the columns stay distinct in L^* (their row-1 entries are 1 and 2), so row 1 keeps v_1, v_2 apart and the multiset is recovered in full. *The algorithm declines to record only what it cannot define, and the tie costs it nothing.* (Had v_1, v_2 been equal in *every* row — a genuinely repeated vector — every piece separating them would carry gap 0 and drop, and the canonical form would simply carry two identical columns: the repeat, recorded faithfully.)

6.3 Why two subdivisions?

We can now see why this sort of embedding needs two subdivisions. The pivot is that the recovery of Lemma 6.1 needs cumulative matrices with integer entries *greater than* 1 — a genuine staircase — and a staircase forms only when the matrices one sums together *overlap*. There are two tempting single-subdivision shortcuts, and each founders on this.

One subdivision, row-local. Canonicalising the row-local pieces on their own quotients by the product group S_n^k — a private permutation per row — which is strictly coarser than the diagonal S_n a multiset allows. Example 4.3 exhibited two distinct multisets it identifies.

One subdivision, cross-row. One might instead sort all nk entries at once and take running sums: a single *cross-row* subdivision. This *does* cumulate, and its matrices *do* span the rows, so it records genuine alignment (enough, for instance, to separate the A, B of Example 4.3). But it cumulates *single cells*, which never overlap, so every running sum stays a flat 0/1 indicator and no staircase ever forms. Run the M of Example 5.2 this way; the surviving cumulative matrices are

$$\left[\begin{smallmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right], \quad \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right], \quad \left[\begin{smallmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \end{smallmatrix}\right], \quad \left[\begin{smallmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{smallmatrix}\right] \qquad \text{(gaps } 4, 2, 2, 1),$$

every entry 0 or 1. The richest, $\left[\begin{smallmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{smallmatrix}\right]$, records only *which* cells are large, as a multiset of columns; contrast the two-stage $L_4 = \left[\begin{smallmatrix} 2 & 0 & 1 \\ 1 & 2 & 0 \end{smallmatrix}\right]$ of Example 5.2, whose 2's encode the within-row order. A lone cross-row subdivision thus records alignment, but *lossily*: its richest prefix is near all-ones, hence content-free, so no single matrix pins a consistent column labelling, and canonicalising the flat prefixes loses what is left. (Distinct multisets that it fails to separate can indeed be exhibited.)

Two suffice: build overlap, then cumulate it. The cure is to make the summands overlap *before* cumulating. That is exactly what the first (per-row) subdivision does: it replaces single cells by nested prefixes top-1 $\subset$ top-2 $\subset \cdots$, which overlap heavily. The second subdivision then cumulates *those*, so shared columns accrue counts above 1 and the staircase appears. Now Lemma 6.1 bites: the richest L^* climbs to counts $n-1$ and (Theorem 6.5) pins the whole column alignment in that one matrix. (Whether the *first* sort is per-row or global is only a matter of tidiness — a global sort followed by a cumulation also works; what is indispensable is the second, cumulating, subdivision.)

Not three. After the second subdivision the data is already injective and continuous; there is no third sorting ambiguity left to launder, so a further subdivision would buy nothing.

A sanity check. For $k = 1$ there are no distinct rows to entangle: the pieces all sit in the one row, "size" alone names them, and the second subdivision collapses to bookkeeping — consistent with Section 3, where one subdivision already sufficed. The second subdivision is precisely the device that the step from $k = 1$ to $k \geq 2$ demands.

7 From the data to a point of $\mathbb{R}^N$: hashing

The output (5) is a list of (canonical matrix, weight) pairs plus k anchors. To land in a single $\mathbb{R}^N$ we compress it; this last step is routine, and we keep it brief.

Fix, once and for all, a "random" rule h that assigns to each possible canonical matrix C a point $h(C) \in \mathbb{R}^D$ (in practice: feed the entries of C to a hash function and use the result as a seed for D pseudo-random coordinates). Then send

$$f(M) \;\longmapsto\; \Big(m_1, \ldots, m_k \,;\; \sum_{s:\,\alpha_s > 0} \alpha_s \, h(\widehat{L}_s) \Big) \;\in\; \mathbb{R}^{k+D}. \tag{6}$$

Continuity is clear: the anchors and weights vary continuously and the points $h(\widehat{L}_s)$ are fixed, so the sum moves continuously (and a pair entering or leaving the sum does so with weight $\alpha_s \to 0$). Invariance is inherited from (5). Only injectivity of this compression needs a word.

Proposition 7.1 (hashing is generically injective)**.** *There are at most $m = k(n-1)$ pairs in any output, so each $\sum_s \alpha_s h(\widehat{L}_s)$ lies in the cone spanned by at most m of the hashed points. If the points $h(C)$ are in general position in $\mathbb{R}^D$ with $D = 2m+1$, then no two* different *such weighted sums coincide: the sum determines both which matrices $\widehat{L}_s$ occur and with what weights α_s. Hence (6) loses nothing that (5) held, and Theorem 6.5 applies.*

The dimension count is the familiar one: an m-term non-negative combination carries m "which-vertex" choices and m continuous weights, and two such m-dimensional cones sit disjointly in general position once $D > 2m$, i.e. $D = 2m+1$. We do not dwell on it: unlike the injectivity of Section 6, it is a standard general-position fact and contains none of the difficulty special to multisets. (One can make "general position" precise — the bad choices of h form a measure-zero set — but for our purposes it is enough that a generic, e.g. random, h works.)

8 Summary

We set out to embed a multiset of n vectors in $\mathbb{R}^k$ — the columns of a matrix M, read without order — as a single point of $\mathbb{R}^N$, demanding permutation-invariance, injectivity and continuity at once. The method:

1. **First subdivision (within each row).** Sort each row and re-express it through gaps and nested prefixes (Section 3). This is the barycentric subdivision of a simplex (Section 2), and it makes the within-row sorting continuous.

2. **Second subdivision (across the pooled pieces).** Pool the prefixes from all rows, sort them by gap, and form cumulative cross-row matrices L_s with weights α_s (Section 5). The same subdivision, one level up.

3. **Canonicalise and hash.** Sort the columns of each L_s and hash the result to a point; output the weighted sum, with the symmetric anchors alongside (Sections 5, 7).

Invariance and continuity were essentially free (Proposition 5.3). The content was injectivity, and there the story is geometric and sharp:

- A *single* subdivision canonicalises objects that do not remember enough. Taken per row it quotients by the too-large product group S_n^k and collides distinct multisets (Example 4.3); taken as one cross-row sort it cumulates disjoint cells, so its matrices stay flat 0/1 and record the column alignment only lossily (§6.3).

- The decisive feature is integer counts above 1 — a staircase — which arise only when one cumulates *overlapping* pieces. So the first subdivision builds the overlap (nested per-row prefixes) and the second cumulates it into cross-row matrices that *remember their pieces* (Lemma 6.1), each piece named label-freely by its $(\text{row}, \text{size})$. The single richest such matrix then pins the whole column alignment with one common permutation — the *diagonal* S_n a multiset allows (Theorem 6.5).

- Two subdivisions are thus necessary and sufficient: one to lay out the overlapping skeleton, one to cumulate it into recoverable form — the second indispensable precisely because $k \geq 2$ rows share their columns.

The elegance the geometry buys is that "injectivity" becomes a sentence about simplices: *a point lies in a unique cell of a subdivided simplex, with unique coordinates there, and identifying cells by the symmetry of their vertices is exactly the multiset symmetry — provided the subdivision is fine enough that one common relabelling, not a private one per row, does the identifying.* Making it "fine enough" is the second barycentric subdivision.